Compilation and System Calls

CS3281 / CS5281

Spring 2026

Tel (615) 343-7472 | Fax (615) 343-7440
1025 16th Avenue South Nashville, TN 37212
www.isis.vanderbilt.edu

VANDERBILT UNIVERSITY

Overview

- C Language Overview
- Compilation Basics: From Source to Executable
- Concepts and Definitions
- Introduction to System Calls

C Language Overview

Tel (615) 343-7472 | Fax (615) 343-7440
1025 16th Avenue South Nashville, TN 37212
www.isis.vanderbilt.edu

C vs C++: A Comparison

	C	C++
Paradigm	Procedural	Object-Oriented
Keywords	32	63
Inheritance	Not Supported	Supported
Polymorphism	Not Supported	Functions, Templates
Allocation	malloc, free	new, delete
Encapsulation	struct	class, struct
Access Control	None	Access Modifiers
Compatibility		Superset of C
Applications	OS, device drivers	Gaming, Web

C vs C++: Compatibility

- C++ is fully backward compatible with C
  - C++ can directly call C code
  - C code can call C++ code that has been declared with `extern "C"`
- Shared structure, grammar, and syntax
- C++ is a superset of C
  - Commonly referred to as "C with class"
- C is historically used in OS kernels and device drivers

C Language Overview

- `struct`
  - Groups variables into a single object
  - Stored as a contiguous block of memory
  - Identical to a C++ `class` with a `public` access modifier
  - May contain variables of any data type
  - Multiple ways of instantiating objects
  - **Must** include the "struct" keyword when declaring variables of corresponding type

```
struct example {
    int var1, var2;
    char var3;
};
struct example e;
e.var1 = 1;
e.var2 = 2;
e.var3 = 'c';

struct example ex1 = {
    1, 2, 'c' };

struct example ex2 = {
    .var2 = 2,
    .var1 = 1,
    .var3 = 'c'
};
```

C Language Overview

- Pointer
  - Stores the **memory address** of a variable
  - Also known as a **reference** to a variable
  - Declare pointer using "*" symbol
  - Obtain variable address using "&"
  - Dereference pointer using "->" or "*"
  - Ex:
    - `ex2->var1 = 1;`
    - What does ex2->var2 contain?

```
struct test {
   int var1, var2;
   char var3;
};

struct test ex1 = {
   1, 2, 'c' };

struct test *ex2 = &ex1;
```

C Language Overview

- Function Pointer
  - Exact same as a variable pointer
  - Contains the address of a function
  - Can be stored in a variable and passed to other functions
  - See example on right for syntax
- **All pointers** are *typically* stored in a memory block of the same size as the underlying hardware architecture
  - E.g., A 64-bit OS uses 64-bit pointers

```
int func(int a, char b)
{
   return a + b;
};

// Function pointer decl
int (*fp)(int, char);

// Function assignment
fp = &func;
```

C Language Overview

- Header Files
  - Have the extension ".h"
  - Used to *declare* (not *define*) types, variables, and definitions
- Compilation Unit
  - Originates from a **single** source file with the extension ".c"
  - Includes contents of **all** "#include"ed header files
  - All Compilation Units linked together during final stage of compilation
- `extern` keyword
  - Declares variables that are **defined** in a different Compilation Unit

Compilation: From Source Code to Runnable Program

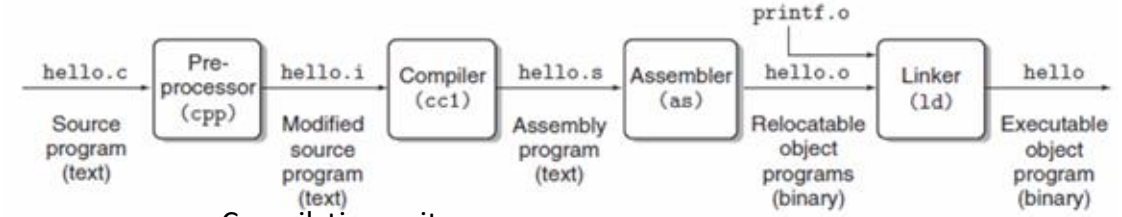
Tel (615) 343-7472 | Fax (615) 343-7440
1025 16th Avenue South Nashville, TN 37212
www.isis.vanderbilt.edu

From Source to Running Program in C

- We begin with a source file saved as text (e.g., prog.c)
  - This is a sequence of bits organized into 8-bit chunks called bytes
    - Each byte represents a character
  - ASCII/UTF-8: Unique byte-sized integer value for each character
    - Source file stored as sequence of bytes
    - Files consisting exclusively of ASCII or UTF-8 characters are text files; binary files otherwise
  - All files are just bits
    - Distinguishing difference is the context in which we view them

From Source to Running Program

- Before it can run, source code must be translated into a sequence of machine-language instructions
  - Packaged into a form called an “executable object program” and stored as a binary file on disk
- Translation is done by a **compiler**
- Individual steps of compilation can be seen with:
  - gcc -E hello.c -o hello.i // produces a modified source program
  - gcc -S hello.i // produces an assembly language program
  - gcc -c hello.s // produces a binary object file
  - gcc hello.o -o hello // produces a linked executable



Compilation unit

From Source to Running Program

- Running “hello world”
  - The shell then loads the executable “hello” file by executing instructions that copy the code and data in the “hello” object file into main memory.
  - Once the code and data are in memory, the OS switches to the “hello” process and begins executing its instructions

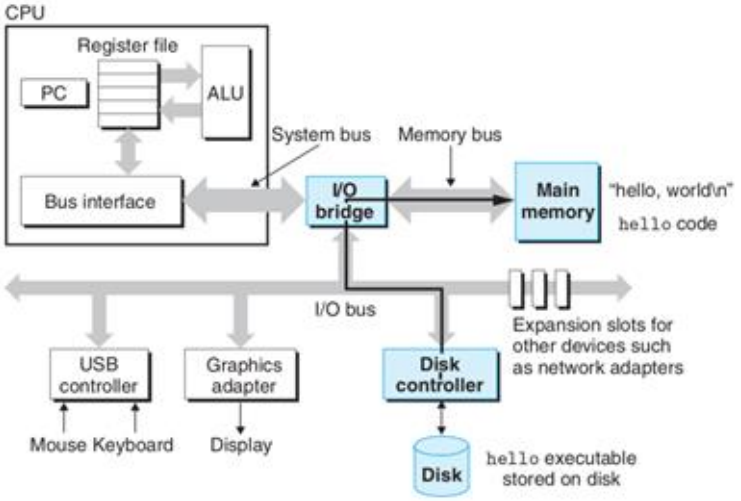
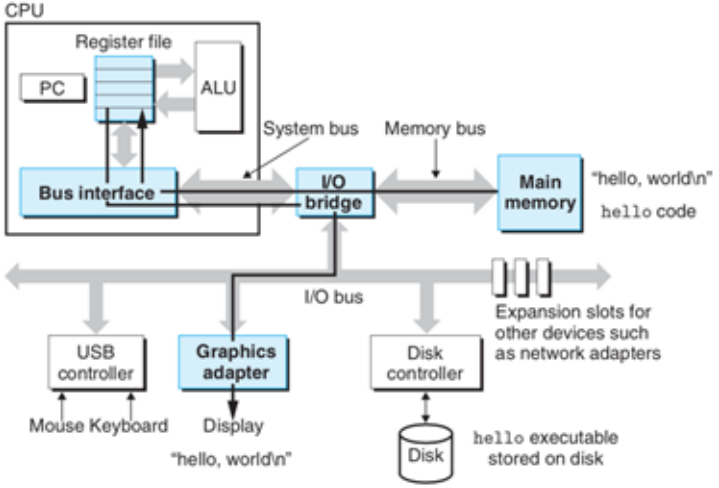


Figure 1.6 Loading the executable from disk into main memory.

From Source to Running Program

- Running “hello world”
- These instructions copy the data bytes “hello, world\n” from main memory to a register
- From there they go to the display device where they are displayed on the screen

Figure 1.7
Writing the output string from memory to the display.



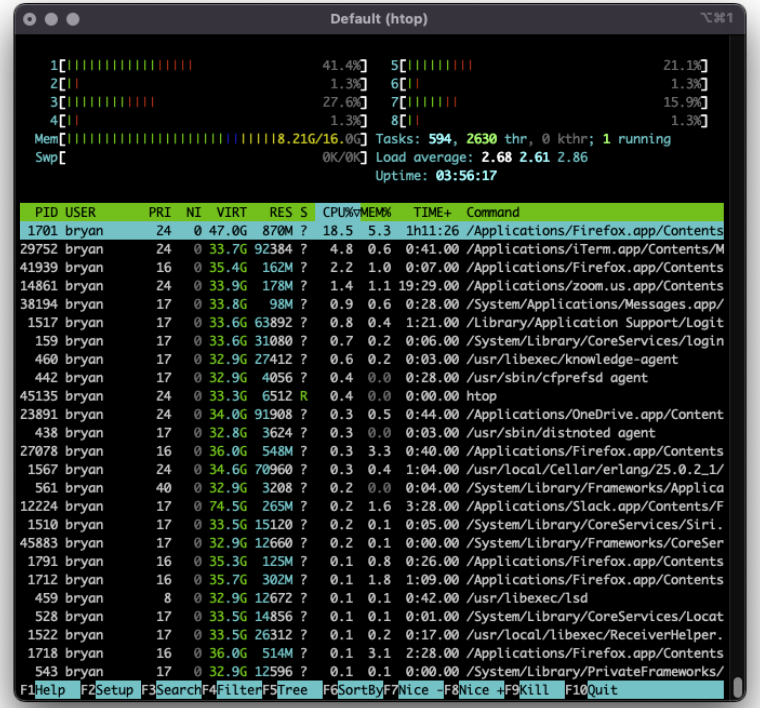
What is an Operating System?

Operating System: Definition

- Many definitions
  - A program that shares a computer among multiple programs and provides a more useful set of services than the hardware alone
  - A program that makes the hardware do "something useful"
- Accomplishes this by "virtualizing" the hardware
  - The OS makes it appear as though every process has the hardware all to itself
- Provides services through a *system call* interface
- The core part of the OS is called the *OS kernel*, or just *kernel*
- Key point: the operating system is itself a program!
  - But unlike other programs, it should have full access to <u>all</u> resources! How can we accomplish this?

Processes

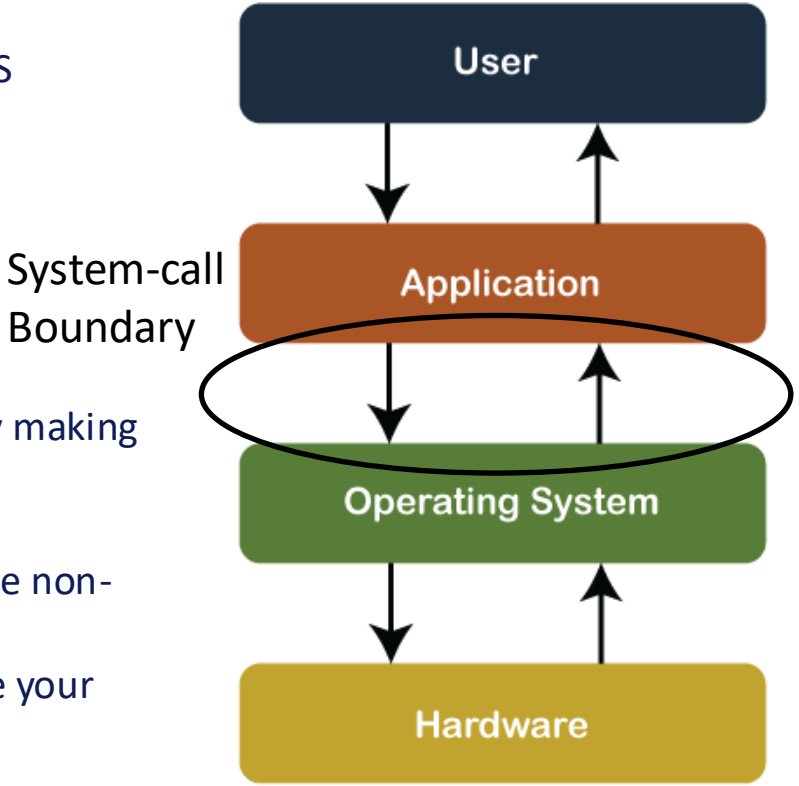
- A process is a running instance of a program
- It's the main unit of **abstraction** provided by the OS
- OS makes your machine easy to use even though many processes run in the machine
- OS does it through **virtualization**
  - Each process thinks it has the whole CPU
  - Each process thinks it has all memory to itself
- Virtualization provides (memory) isolation:
  - Prevents a process X from corrupting or spying on a process Y
  - Prevents a process from corrupting the operating system itself



"ps" "top" or "htop" show all active processes

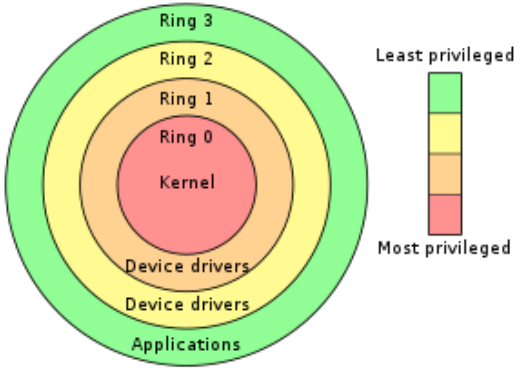
System Calls

- The system-call API is the interface that makes OS services available to user-level programs
- What are examples of these services?
  - Create a process (running program)
  - Request memory
  - Read/write from/to a file
  - Send data over the network
- A process requests these services from the OS by making a *system call*
- Why does the OS provide these services?
  - User-level processes cannot be trusted to be non-malicious
  - Example: one malicious process could erase your entire hard drive!



Rings: Kernel Mode vs. User Mode

- The kernel needs full access to all hardware and CPU instructions
- User-level processes (like Chrome or Firefox) should <u>not</u> be allowed full access to all hardware <u>nor</u> should they be able to execute all CPU instructions
- How should we enforce this? With hardware support!
  - A flag in a CPU register (status register) determines whether privileged instructions are allowed
  - On x86 it's called the CPL (current privilege level)
    - CPL = 0 (0b00) means kernel mode: privileged
    - CPL = 3 (0b11) means user mode: no privilege
  - The CPL enforces isolation in several ways:
    - Guards access to the privilege register
    - Checks every memory read/write
    - Checks every I/O port access



- In summary:
  - The privilege level determines whether instructions can access privileged hardware
  - Only the kernel should be allowed to operate in a privileged mode

Invoking a System Call

- How and when should the privilege level change?
- When a system call is invoked:
  - A special instruction (supervisor call) sets CPL=0
  - Execution jumps to a specific entry point in the kernel, which can then do further validation
  - The system call return sets CPL=3 before returning to user code

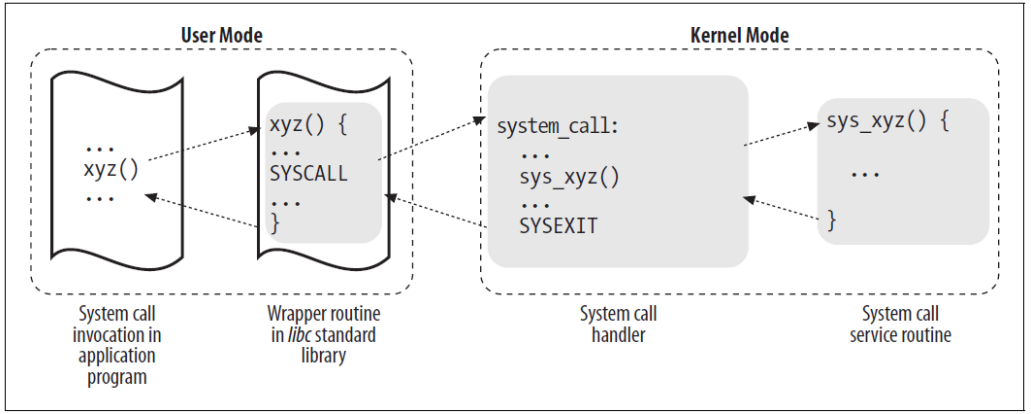


Figure 10-1. Invoking a system call

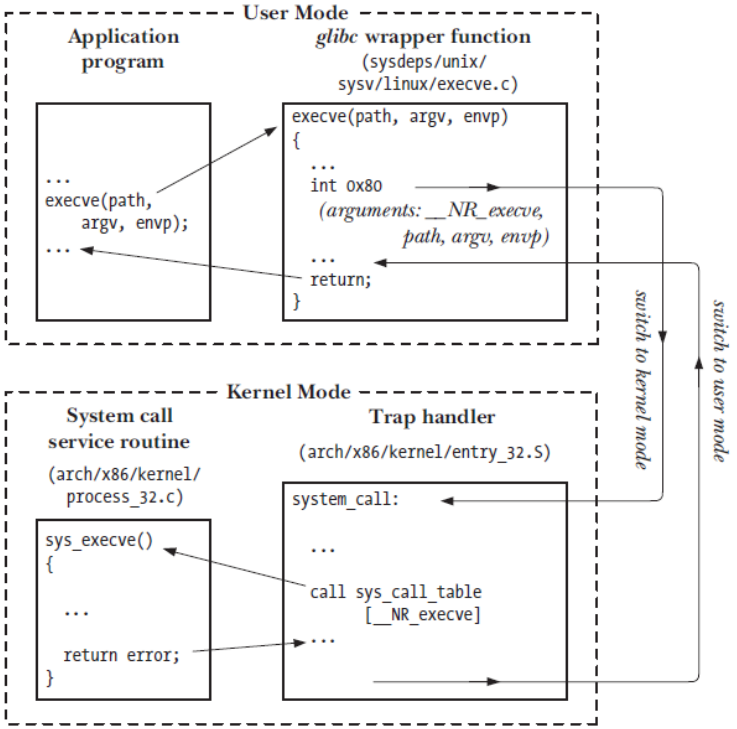
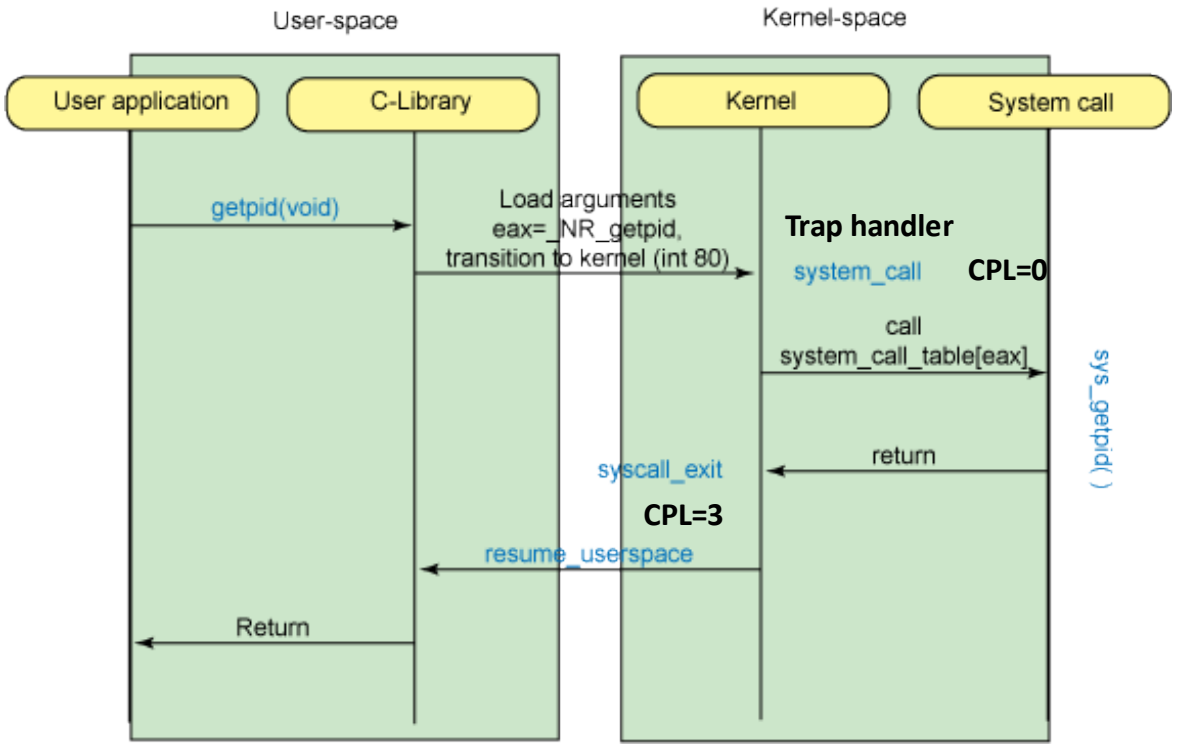


Figure 3-1: Steps in the execution of a system call

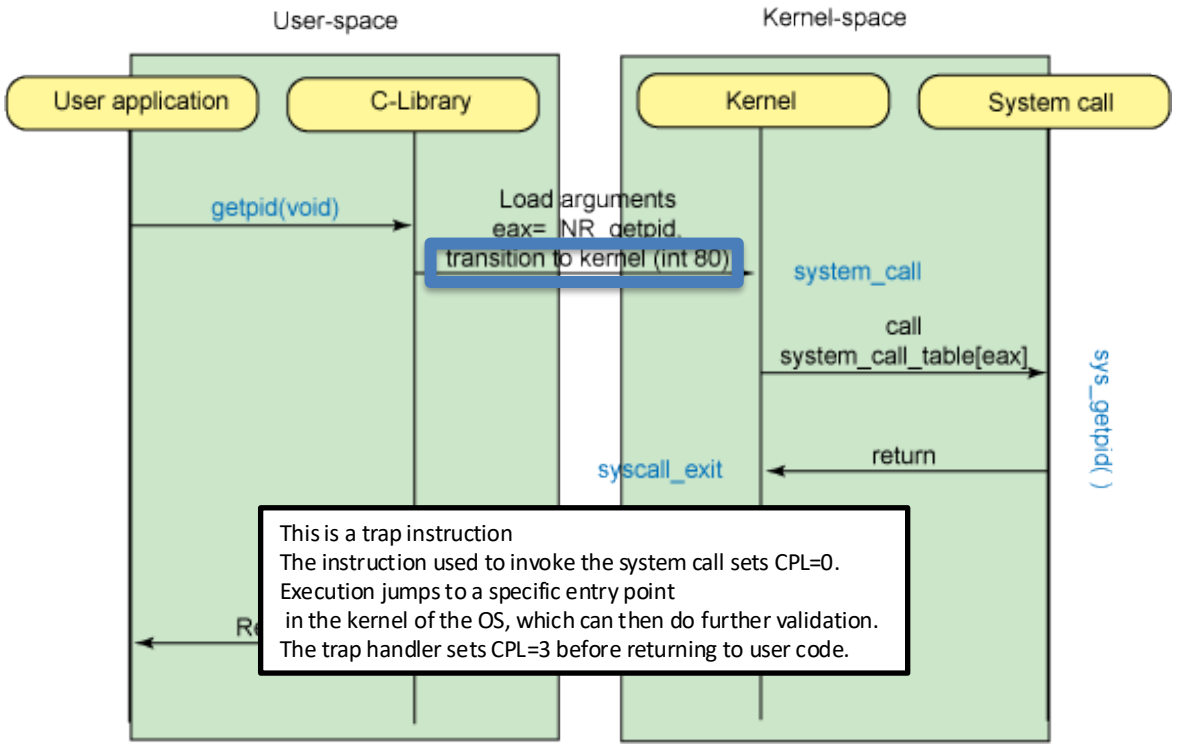
Details of System-Call Implementation

- The program invokes a wrapper function in the C library. The reason for this is that the wrapper function sets up the system-call arguments as expected by the OS
- The wrapper function puts any arguments to the system call on the stack
- All system calls enter the kernel the same way, so the kernel must be able to identify them. In x86, the wrapper function in the C library copies the system call number into the %eax register of the CPU
- The wrapper function executes a *trap* machine instruction. This is *int 0x80*. This causes the processor to switch from user mode to kernel mode (that is, it sets CPL=0) and executes the code pointed to by location *0x80* of the system's trap (branch) vector. The executed code is a function that handles traps
- The kernel invokes its *system_call* routine (trap handler) to handle the trap to location *0x80*. This is where the meat of the system-call logic happens: the kernel does some bookkeeping, checks the validity of the arguments, invokes the service routine, and finally the service routine returns a result status to the *system_call* routine
- The wrapper function checks if the service returned an error, and if so, sets a global variable named *errno* with this error value. The wrapper function returns to the caller and provides an integer return value to indicate success or failure

Another system-call control flow example



Another system call control flow example



This is a trap instruction
The instruction used to invoke the system call sets CPL=0.
Execution jumps to a specific entry point
in the kernel of the OS, which can then do further validation.
The trap handler sets CPL=3 before returning to user code.

Tracing system calls

- The *strace* command lets you see the system calls that are made by a process
- Example: type “strace ls” at a terminal to see all the system calls that the ls (which lists files in the current directory) makes. You might see calls like:
  - execve -- this loads a new program and starts running it
  - open -- open a file
  - read -- read from a file
  - close -- close a file
  - fstat -- get information about a file

Tracing systems calls

- strace ls

Execv call loads in a new program into a process
- We will learn more about it in the next lecture

```
vagrant@cs281spring2017devbox:~$ strace ls
execve("/bin/ls", ["ls"], [/* 58 vars */]) = 0
brk(NULL)                                   = 0x1164000
access("/etc/ld.so.nohwcap", F_OK)          = -1 ENOENT (No
such file or directory)
…..
open("/etc/ld.so.cache", O_RDONLY|O_CLOEXEC) = 3
fstat(3, {st_mode=S_IFREG|0644, st_size=70675, ...}) = 0
mmap(NULL, 70675, PROT_READ, MAP_PRIVATE, 3, 0) =
0x7f21e9c83000
close(3)
….
exit_group(0)                               = ?
+++ exited with 0 +++                                  = 0
```

The exit_group call terminates a process